

+

[REPORT_ID // DBMS-Project-01]

MULTI-COMPANY CRM SYSTEM

Database Architecture & Relational Implementation Report

PROJECT DEVELOPERS

Muhammad Arslan Nazir —F2024332048

Noor Fatima —F2024332127

Umm e Hani —F2024332141

ACADEMIC SESSION

Course: Database Systems

Instructor: Noreen Ashraf

+

EXECUTIVE OVERVIEW

Scenario: SaaS Multi-Tenant Customer Relationship Management (CRM) Platform.

In the modern data-driven landscape, CRM usage has moved beyond simple contact storage to serving as a centralized system of record. Approximately 91% of companies with more than 11 employees use a CRM, and the vast majority rely on cloud-based (SaaS) solutions. Building isolated databases for every client is technologically inefficient and financially unscalable.

The problem this relational database solves is establishing a secure, highly-normalized, centralized architecture where multiple independent companies can operate simultaneously on a shared infrastructure.

This system guarantees strict tenant data isolation by propagating a persistent `company_id` foreign key across every operational table in the ecosystem.

SYSTEM ARCHITECTURE

NORMALIZATION STANDARD (3NF)

The database schema is strictly normalized up to the Third Normal Form (3NF) to aggressively eliminate data redundancy and ensure transactional integrity across the 12 underlying tables.

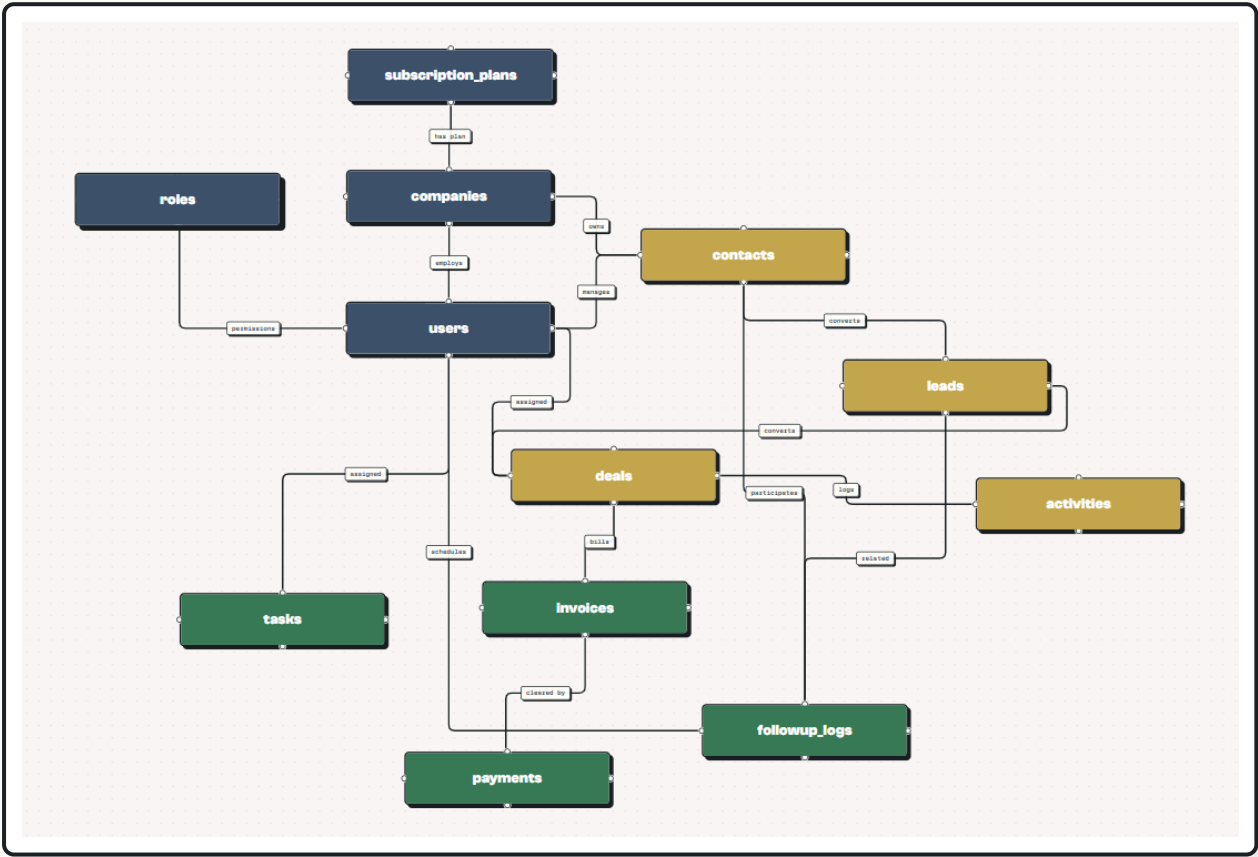
- **1NF:** All attributes contain atomic values. Addresses are split into distinct `city`, `country`, and `address` fields.
- **2NF:** Zero partial dependencies. All non-key attributes are fully functionally dependent on the primary keys.
- **3NF:** Transitive dependencies eliminated. (e.g., `plan_name` resides strictly in `subscription_plans`, connected via `plan_id`, averting duplication inside `companies`).

DATA INTEGRITY ENFORCEMENT

- **Entity Integrity:** `AUTO_INCREMENT` Primary Keys are implemented across all 12 tables.
- **Domain Integrity:** Strict `ENUM` definitions govern state columns to prevent invalid data insertion.
- **Referential Integrity:** Cascading Foreign Key (FK) constraints ensure orphaned records cannot exist.

ENTITY-RELATIONSHIP TOPOLOGY

The mapping below outlines the 12 core tables across the 3 sub-modules. The central routing node is the `companies` table, ensuring robust multi-tenant data segregation.

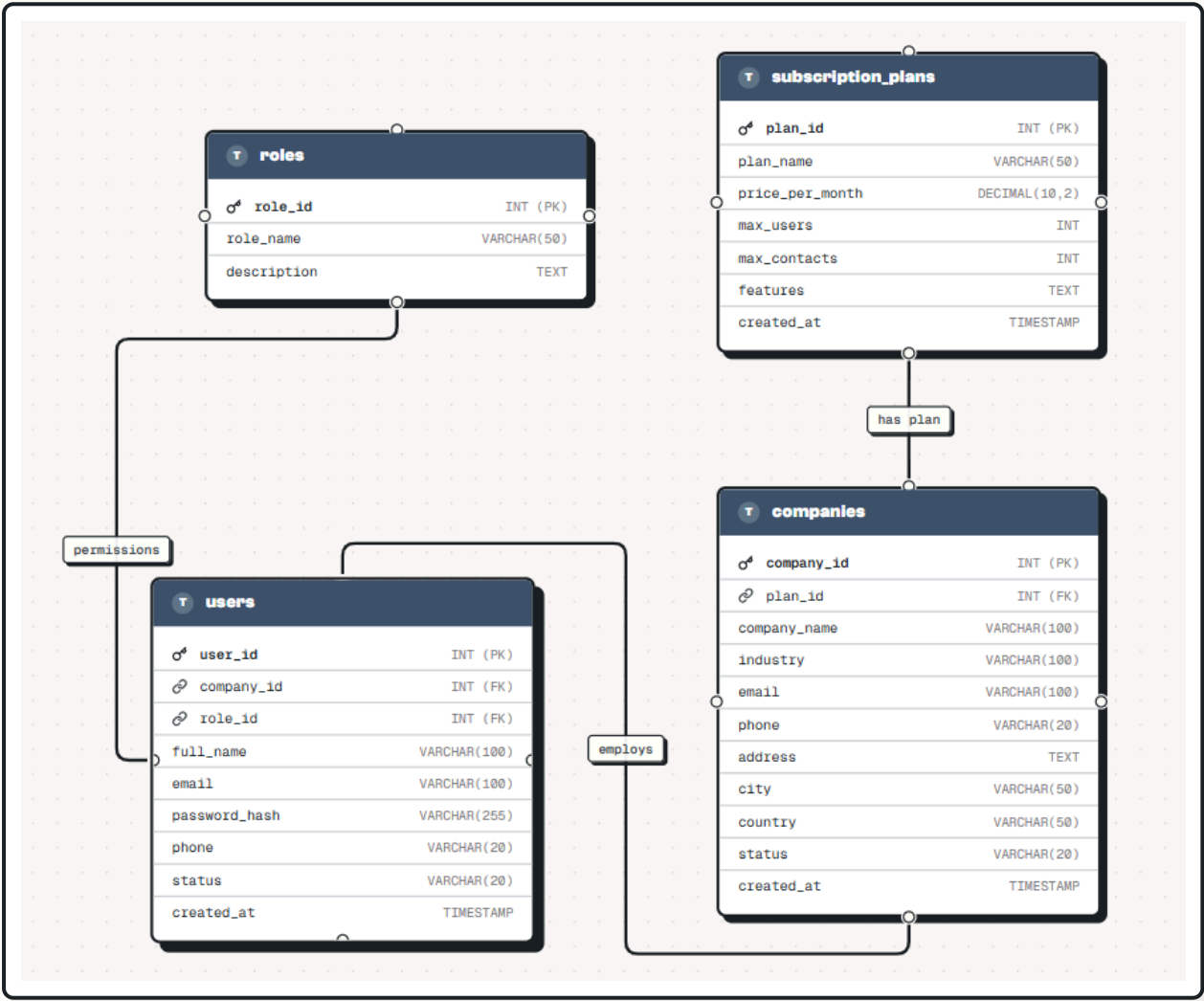


RELATIONAL SCHEMA

The following sections outline the normalized relational schemas for each operational module. For full relational context, refer to the module-specific Crow's Foot ER diagrams embedded within each section.

MODULE 1: CORE TENANCY & PROVISIONING

This module manages user identities, access control roles, multi-tenant companies, and subscription plan tiers.



// 1. SUBSCRIPTION_PLANS

COLUMN NAME	DATA TYPE	CONSTRAINTS
plan_id	INT	PK, AUTO_INCREMENT
plan_name	VARCHAR(50)	NOT NULL, UNIQUE
price_per_month	DECIMAL(10,2)	NOT NULL
max_users	INT	NOT NULL
max_contacts	INT	NOT NULL
features	TEXT	
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

// 2. COMPANIES

COLUMN NAME	DATA TYPE	CONSTRAINTS
company_id	INT	PK, AUTO_INCREMENT
<i>plan_id</i>	INT	FK → subscription_plans
company_name	VARCHAR(100)	NOT NULL
industry	VARCHAR(100)	
email	VARCHAR(100)	NOT NULL, UNIQUE
phone	VARCHAR(20)	
address	TEXT	
city	VARCHAR(50)	
country	VARCHAR(50)	
status	ENUM	('active','suspended','cancelled')
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

// 3. ROLES

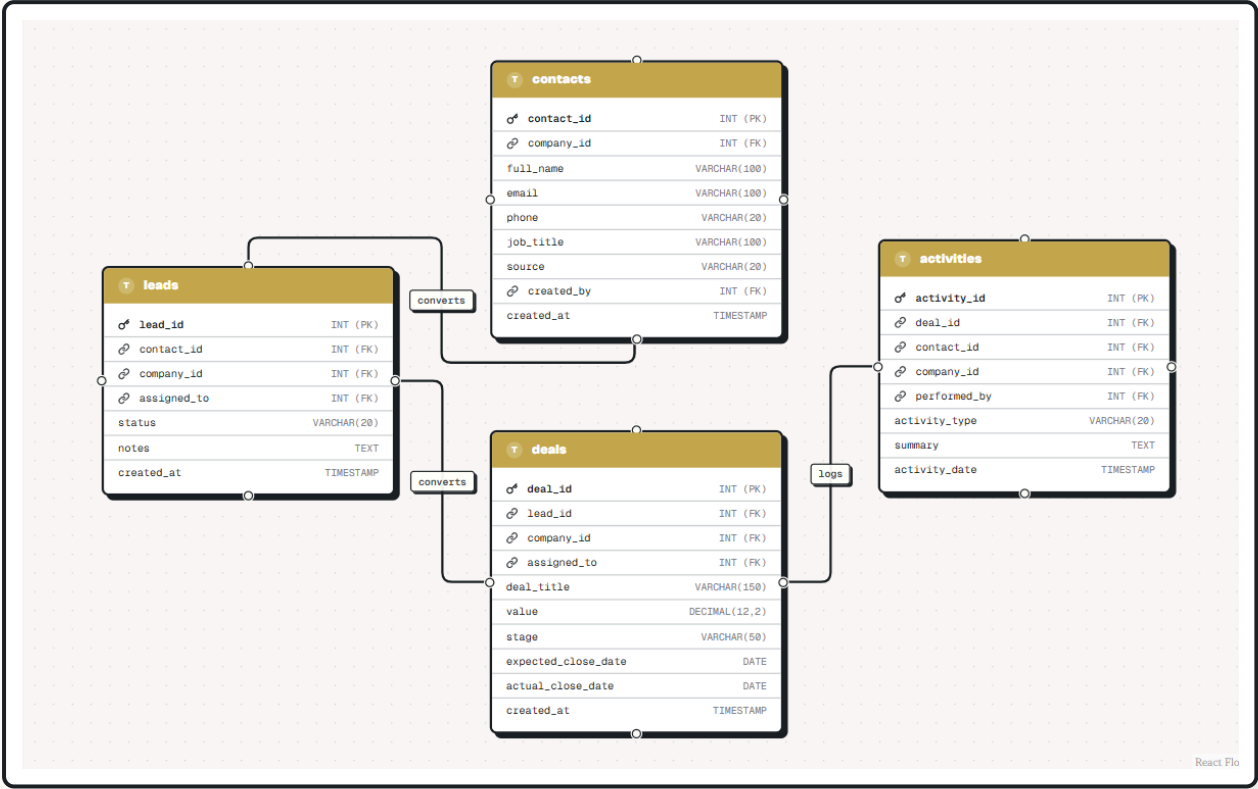
COLUMN NAME	DATA TYPE	CONSTRAINTS
role_id	INT	PK, AUTO_INCREMENT
role_name	VARCHAR(50)	NOT NULL, UNIQUE
description	TEXT	

// 4. USERS

COLUMN NAME	DATA TYPE	CONSTRAINTS
user_id	INT	PK, AUTO_INCREMENT
company_id	INT	FK → companies
role_id	INT	FK → roles
full_name	VARCHAR(100)	NOT NULL
email	VARCHAR(100)	NOT NULL, UNIQUE
password_hash	VARCHAR(255)	NOT NULL
phone	VARCHAR(20)	
status	ENUM	('active','inactive')
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

MODULE 2: SALES PIPELINE & LEAD NURTURING

This module tracks business opportunities, including contacts, leads, business deals, and commercial interaction logs.



// 5. CONTACTS

COLUMN NAME	DATA TYPE	CONSTRAINTS
contact_id	INT	PK, AUTO_INCREMENT
<i>company_id</i>	INT	FK → companies
full_name	VARCHAR(100)	NOT NULL
email	VARCHAR(100)	
phone	VARCHAR(20)	
job_title	VARCHAR(100)	
source	ENUM	('referral','web','cold_outreach','social')
<i>created_by</i>	INT	FK → users
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

// 6. LEADS

COLUMN NAME	DATA TYPE	CONSTRAINTS
lead_id	INT	PK, AUTO_INCREMENT
<i>contact_id</i>	INT	FK → contacts
<i>company_id</i>	INT	FK → companies
<i>assigned_to</i>	INT	FK → users
status	ENUM	('new','contacted','qualified','lost')
notes	TEXT	
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

// 7. DEALS

COLUMN NAME	DATA TYPE	CONSTRAINTS
deal_id	INT	PK, AUTO_INCREMENT
<i>lead_id</i>	INT	FK → leads
<i>company_id</i>	INT	FK → companies
<i>assigned_to</i>	INT	FK → users
deal_title	VARCHAR(150)	NOT NULL
value	DECIMAL(12,2)	
stage	ENUM	('proposal','negotiation','closed_won','closed_lost')
expected_close_date	DATE	
actual_close_date	DATE	
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

// 8. ACTIVITIES

COLUMN NAME	DATA TYPE	CONSTRAINTS
activity_id	INT	PK, AUTO_INCREMENT
<i>deal_id</i>	INT	FK → deals
<i>contact_id</i>	INT	FK → contacts
<i>company_id</i>	INT	FK → companies
<i>performed_by</i>	INT	FK → users
activity_type	ENUM	('call','email','meeting','note')
summary	TEXT	
activity_date	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

MODULE 3: OPERATIONS & TRANSACTIONAL FINANCE

This module governs task lists, follow-up logs, invoice dispatching, and payments collection.



// 9. TASKS

COLUMN NAME	DATA TYPE	CONSTRAINTS
task_id	INT	PK, AUTO_INCREMENT
<i>company_id</i>	INT	FK → companies
<i>assigned_to</i>	INT	FK → users
<i>deal_id</i>	INT	FK → deals, NULL
<i>contact_id</i>	INT	FK → contacts, NULL
title	VARCHAR(150)	NOT NULL
description	TEXT	
due_date	DATE	
priority	ENUM	('low','medium','high')
status	ENUM	('pending','in_progress','completed')
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

// 10. INVOICES

COLUMN NAME	DATA TYPE	CONSTRAINTS
invoice_id	INT	PK, AUTO_INCREMENT
<i>deal_id</i>	INT	FK → deals
<i>company_id</i>	INT	FK → companies
<i>issued_by</i>	INT	FK → users
total_amount	DECIMAL(12,2)	NOT NULL
due_date	DATE	
status	ENUM	('draft','sent','paid','overdue')
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

// 11. PAYMENTS

COLUMN NAME	DATA TYPE	CONSTRAINTS
payment_id	INT	PK, AUTO_INCREMENT
<i>invoice_id</i>	INT	FK → invoices
<i>company_id</i>	INT	FK → companies
amount_paid	DECIMAL(12,2)	NOT NULL
payment_date	DATE	NOT NULL
payment_method	ENUM	('bank_transfer','card','cash','cheque')
notes	TEXT	

// 12. FOLLOWUP_LOGS

COLUMN NAME	DATA TYPE	CONSTRAINTS
followup_id	INT	PK, AUTO_INCREMENT
<i>company_id</i>	INT	FK → companies
<i>contact_id</i>	INT	FK → contacts
<i>lead_id</i>	INT	FK → leads, NULL
<i>deal_id</i>	INT	FK → deals, NULL
<i>scheduled_by</i>	INT	FK → users
scheduled_date	DATETIME	NOT NULL
outcome	ENUM	('reached','no_answer','rescheduled','converted')
next_action	TEXT	
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

SAMPLE QUERIES & RESULTS

These queries demonstrate data retrieval, aggregation, joins, filtering, grouping, and subqueries as per the project requirements.

01. Basic Data Retrieval & Filtering

```
SELECT company_name, email, phone
      FROM companies
      WHERE status = 'active';
```

// SQL_QUERY

COMPANY_NAME	EMAIL	PHONE
Nexus Data Corp	contact@nexusdata.io	+1-555-0198
Aura Logistics	hello@auralogistics.com	+1-555-0245

02. Basic JOIN

```
SELECT users.full_name, roles.role_name
      FROM users
      JOIN roles ON users.role_id = roles.role_id;
```

// SQL_QUERY

FULL_NAME	ROLE_NAME
Elena Rostova	Sales Representative
Julian Vance	System Admin

SAMPLE QUERIES & RESULTS

03. Simple Aggregation

```
SELECTCOUNT(lead_id) AS total_leads
      FROM leads;
```

// SQL_QUERY

TOTAL_LEADS

184

04. Aggregation with Filtering

```
SELECTSUM(value) AS
      total_revenue
      FROM deals
      WHERE stage = 'closed_won';
```

// SQL_QUERY

TOTAL_REVENUE

\$1,157,650.00

SAMPLE QUERIES & RESULTS

05. Grouping (GROUP BY)

```
SELECT company_id, COUNT(user_id) AS user_count
      FROM users
      GROUP BY company_id;
```

// SQL_QUERY

COMPANY_ID	USER_COUNT
1	27
2	14

06. Sorting (ORDER BY)

```
SELECT invoice_id, total_amount, status
      FROM invoices
      ORDER BY total_amount DESC;
```

// SQL_QUERY

INVOICE_ID	TOTAL_AMOUNT	STATUS
INV-9002	\$45,000.00	paid
INV-9003	\$12,500.00	draft

SAMPLE QUERIES & RESULTS

07. Basic Subquery

```
SELECT deal_title, value
      FROM deals
      WHERE company_id = (
        SELECT company_id
        FROM companies
        WHERE company_name = 'Nexus Data Corp'
      );
```

// SQL_QUERY

DEAL_TITLE	VALUE
Q3 Server Fleet Upgrade	\$125,000.00
Cloud Storage Expansion	\$45,000.00

08. JOIN with Filtering

```
SELECT tasks.title, tasks.priority, users.full_name
      FROM tasks
      JOIN users ON tasks.assigned_to = users.user_id
      WHERE tasks.status = 'pending';
```

// SQL_QUERY

TITLE	PRIORITY	FULL_NAME
Finalize Q3 Contract	high	Julian Vance
Send Compliance PDF	medium	Elena Rostova

SAMPLE QUERIES & RESULTS

09 . Grouping with HAVING

// SQL_QUERY

```
SELECT company_id, COUNT(user_id) AS total_users
      FROM users
      GROUP BY company_id
      HAVINGCOUNT(user_id) > 5;
```

COMPANY_ID	TOTAL_USERS
1	27
2	14

10 . Multiple JOINS

// SQL_QUERY

```
SELECT contacts.full_name, deals.deal_title, activities.activity_type
      FROM activities
      JOIN contacts ON activities.contact_id =
      contacts.contact_id
      JOIN deals ON activities.deal_id =
      deals.deal_id;
```

FULL_NAME	DEAL_TITLE	ACTIVITY_TYPE
Sylvia Plath	Q3 Server Fleet Upgrade	meeting
Marcus Vance	Cloud Storage Expansion	email